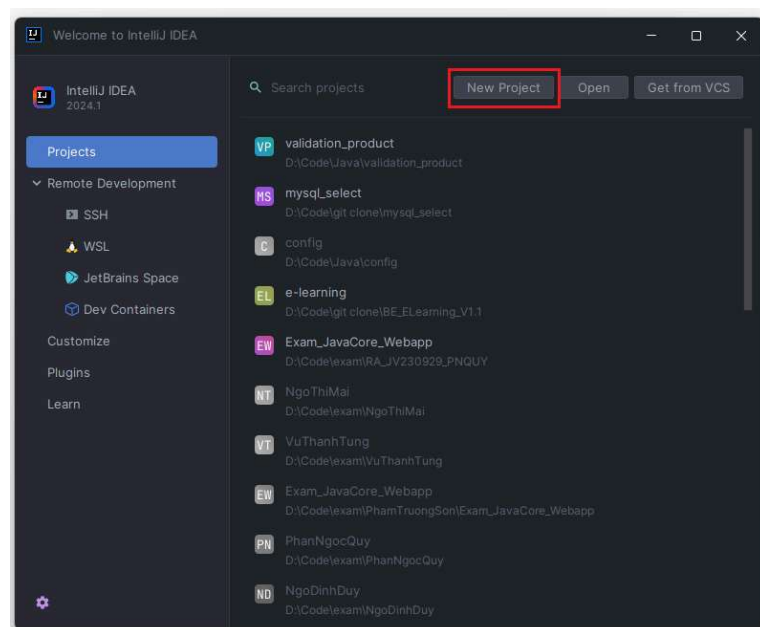


🕒 Cập nhật tháng 8 năm 2024

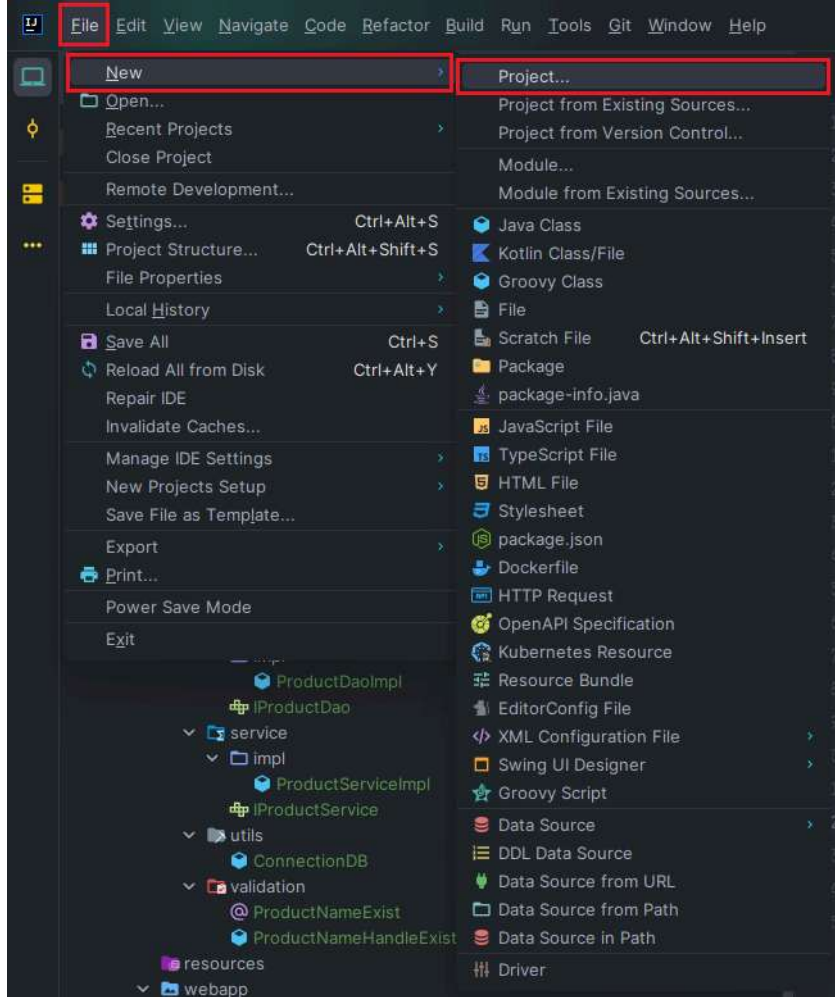
[Bài đọc] Demo thiết kế các API cơ bản vs JWT

- Bài toán chúng ta đặt ra làm sao để bảo mật được hệ thống, thông tin khách hàng, chúng ta sẽ sử dụng thư viện Spring Security
- Chúng ta làm việc với đối tượng users và roles như sau:
- Đối tượng users:
 - id : Long
 - fullName: String
 - email: String
 - password: String
 - address: String
 - phone: String
- Đối tượng roles:
 - id: Long
 - RoleName: enum (ROLE_ADMIN, ROLE_USER)
- 2 đối tượng users và roles là nhiều với nhiều (n-n)
- Yêu cầu chức năng:
 - Đăng ký
 - Đăng nhập

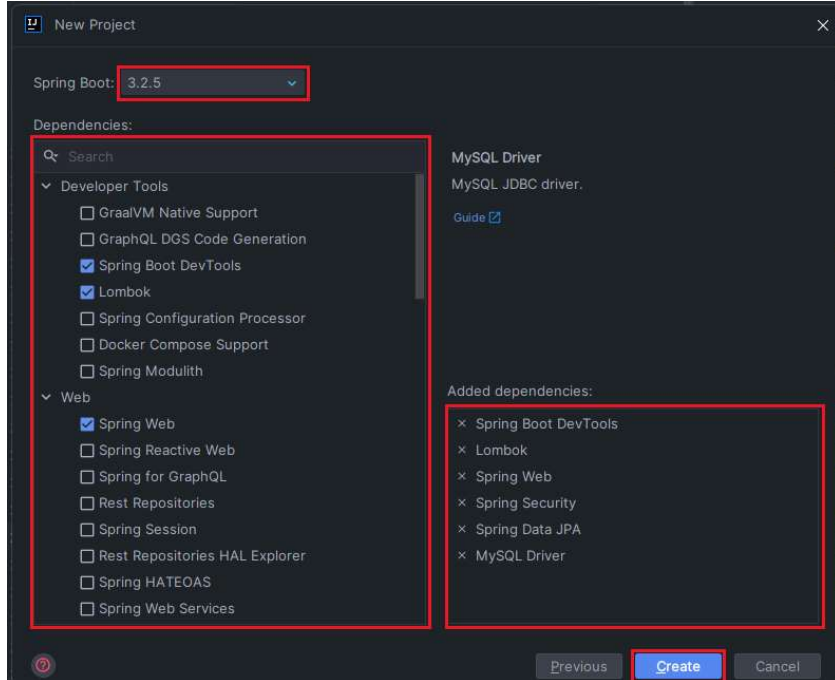
Bước 1: Khởi tạo dự án Spring Boot và thêm những thư viện cần thiết như: **Spring Boot Devtool, Lombok, Spring Web, Spring Data JPA, MySQL Driver, Spring Security,**...



Hoặc :

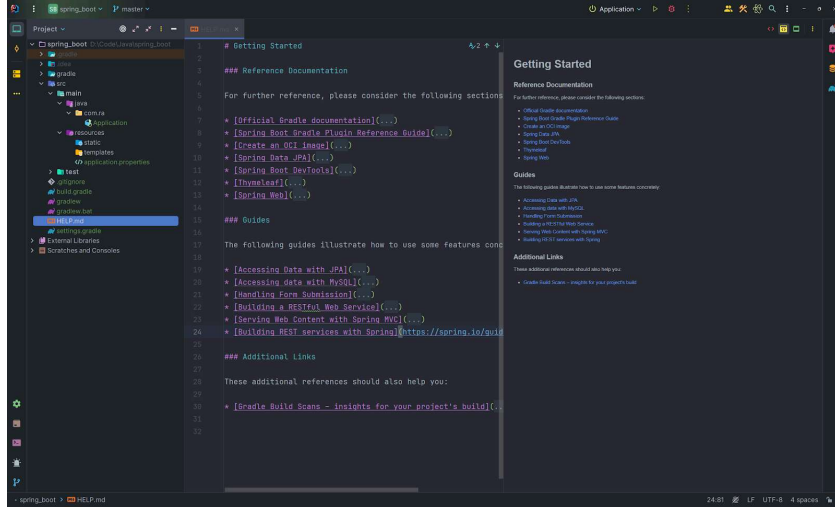


Bước 2: màn hình sẽ hiển thị ra lựa chọn **Spring Boot** hoặc bản IntelliJ cũ hơn sẽ là (**Spring Initilizer**)



- Bấm Next để đến bước thêm thư viện:
- Sẽ phải lựa chọn version (phiên bản) dự án spring boot ở bên trái (**dependencies**), bên phải là những thư viện đã được thêm vào dự án, bấm vào Create để tạo dự án **Spring Boot** đầu tiên

Bước 3: Khi dự án chạy xong dự án sẽ như sau:



Sẽ cần thiết thêm những thư viện bên ngoài như:

```
implementation group: 'io.jsonwebtoken', name: 'jjwt', version: '0.9.1'
implementation group: 'javax.xml.bind', name: 'jaxb-api', version: '2.3.1'
```

vào file build.gradle để đủ thư viện jwt và javax.xml.bind để có thể generate được token

Bước 4: Cấu hình kết nối với database và jpa và security như sau:

```
spring.datasource.url=jdbc:mysql://localhost:3306/security_jwt?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=123456
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jooq.sql-dialect=org.hibernate.dialect.MySQL8Dialect
```

```
spring.jpa.hibernate.naming.implicit-
strategy=org.hibernate.boot.model.naming.ImplicitNamingStrategyLegacyJpaImpl
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
```

```
spring.jpa.properties.hibernate.show_sql=true
spring.jpa.properties.hibernate.format_sql=true
```

```
jwt.secret_key=rikkei_academy
jwt.expired=60000
```

Bước 5: Tạo cấu trúc thư mục dự án như sau:

Bước 6: Tạo đối tượng roleName ở package constants như sau:

```
public enum RoleName {
    ROLE_ADMIN,
    ROLE_USER
}
```

Bước 7: Tạo các thực thể như yêu cầu của đề bài là users và roles như sau:

- Roles:

```
import com.ra.constants.RoleName;
import jakarta.persistence.*;
import lombok.*;

@Entity
@NoArgsConstructor
@AllArgsConstructor
@Getter
@Setter
@Builder
public class Roles {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Enumerated(EnumType.STRING)
    private RoleName roleName;
}
```

- Users:

```
@Entity
@NoArgsConstructor
```

```

@AllArgsConstructor
@Getter
@Setter
@Builder
public class Users {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String fullName;
    private String email;
    private String password;
    private String address;
    private String phone;
    @ManyToMany
    @JoinTable(
        name = "user_role",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<Roles> roles;
}

```

Bước 8: Tạo các repository cho từng đối tượng như userRepository và roleRepository như sau:

- IRoleRepository:

```

import com.ra.constants.RoleName;
import com.ra.model.Roles;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface IRoleRepository extends JpaRepository<Roles, Long> {
    Optional<Roles> findByRoleName(RoleName roleName);
}

```

- IUserRepository:

```

import com.ra.model.Users;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface IUserRepository extends JpaRepository<Users, Long> {
    Optional<Users> findByEmail(String email);
}

```

Bước 8: Cấu hình Security và Jwt cho dự án như sau:

- UserPrincipal:

```

import com.ra.model.Users;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.util.Collection;

@NoArgsConstructor
@AllArgsConstructor
@Data
@Builder
public class UserPrincipal implements UserDetails {
    private Long id;
    private String fullName;
    private String email;
    private String password;
    private String address;
    private String phone;
    private Collection<? extends GrantedAuthority> authorities;
}

```

```

        public static UserDetails getUserDetails(Users users) {
            return UserPrincipal.builder()
                .id(users.getId())
                .fullName(users.getFullName())
                .email(users.getEmail())
                .password(users.getPassword())
                .address(users.getAddress())
                .phone(users.getPhone())
                .authorities(users.getRoles().stream().map(role -
> new SimpleGrantedAuthority(role.getRoleName().name()).toList())
                .build());
        }

        @Override
        public Collection<? extends GrantedAuthority> getAuthorities() {
            return this.authorities;
        }

        @Override
        public String getPassword() {
            return this.password;
        }

        @Override
        public String getUsername() {
            return this.email;
        }

        @Override
        public boolean isAccountNonExpired() {
            return true;
        }

        @Override
        public boolean isAccountNonLocked() {
            return true;
        }

        @Override
        public boolean isCredentialsNonExpired() {
            return true;
        }

        @Override
        public boolean isEnabled() {
            return true;
        }
    }
}

```

- UserDetailServiceCustom:

```

import com.ra.model.Users;
import com.ra.repository.IUserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
public class UserDetailServiceCustom implements UserDetailsService {
    @Autowired
    private IUserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        Users users = userRepository.findByEmail(username).orElseThrow(() -
> new UsernameNotFoundException("username not found"));
        return UserPrincipal.getUserDetails(users);
    }
}

```

- JwtProvider:

```

import com.ra.security.user_principal.UserPrincipal;
import io.jsonwebtoken.*;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;

```

```

import org.springframework.stereotype.Component;

import java.util.Date;

@Component
@Slf4j
public class JwtProvider {
    @Value("${jwt.secret_key}")
    private String SECRET_KEY;
    @Value("${jwt.expired}")
    private Long EXPIRED;

    public String generateToken(UserPrincipal userPrincipal) {
        return Jwts.builder().setSubject(userPrincipal.getUsername())
            .setIssuedAt(new Date())
            .setExpiration(new Date(new Date().getTime() + EXPIRED))
            .signWith(SignatureAlgorithm.HS512, SECRET_KEY)
            .compact();
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parser().setSigningKey(SECRET_KEY).parseClaimsJwt(token);
            return true;
        } catch (ExpiredJwtException e) {
            log.error("Failed -> Expired Token Message {}", e.getMessage());
        } catch (UnsupportedJwtException e) {
            log.error("Failed -> Unsupported Token Message {}", e.getMessage());
        } catch (MalformedJwtException e) {
            log.error("Failed -> Invalid Format Token Message {}", e.getMessage());
        } catch (SignatureException e) {
            log.error("Failed -> Invalid Signature Token Message {}", e.getMessage());
        } catch (IllegalArgumentException e) {
            log.error("Failed -> Claims Empty Token Message {}", e.getMessage());
        }
        return false;
    }

    public String getUsernameFromToken(String token) {
        return Jwts.parser().setSigningKey(SECRET_KEY).parseClaimsJwt(token).getBody().getSubject();
    }
}

```

- **JwtTokenFilter:**

```

import com.ra.security.user_principal.UserDetailServiceCustom;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
@Slf4j
@RequiredArgsConstructor
public class JwtTokenFilter extends OncePerRequestFilter {
    private final JwtProvider jwtProvider;
    private final UserDetailServiceCustom userDetailServiceCustom;

    @Override
    protected
    void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain) throws
        try {
            String token = getTokenFromRequest(request);
            if (token != null && jwtProvider.validateToken(token)) {
                String username = jwtProvider.getUsernameFromToken(token);
                if (username != null) {

```

```

        UserDetails userDetails = userDetailsServiceCustom.loadUserByUsername(username);
        Authentication authentication = new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());
        SecurityContextHolder.getContext().setAuthentication(authentication);
    }
}
} catch (Exception e) {
    log.error("Un Authentication {}", e.getMessage());
}
}
filterChain.doFilter(request, response);
}

```

```

public String getTokenFromRequest(HttpServletRequest request) {
    String header = request.getHeader("Authorization");
    if (header != null && header.startsWith("Bearer ")) {
        return header.substring(7);
    }
    return null;
}
}

```

- **JwtEntryPoint:**

```

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.stereotype.Component;

```

```
import java.io.IOException;
```

```
@Component
```

```
@Slf4j
```

```
public class JwtEntryPoint implements AuthenticationEntryPoint {
```

```
    @Override
```

```
    public
```

```
void commence(HttpServletRequest request, HttpServletResponse response, AuthenticationException authException) throws
```

```
    log.error("Un Authentication {}", authException.getMessage());
```

```
    response.setStatus(HttpStatus.UNAUTHORIZED.value());
```

```
    response.getWriter().write("Authentication Failed");
```

```
    }
```

```
}
```

- **AccessDenied:**

```

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.web.access.AccessDeniedHandler;
import org.springframework.stereotype.Component;

```

```
import java.io.IOException;
```

```
@Component
```

```
@Slf4j
```

```
public class AccessDenied implements AccessDeniedHandler {
```

```
    @Override
```

```
    public
```

```
void handle(HttpServletRequest request, HttpServletResponse response, AccessDeniedException accessDeniedException) throws
```

```
    log.error("Un Permission {}", accessDeniedException.getMessage());
```

```
    response.setStatus(HttpStatus.FORBIDDEN.value());
```

```
    response.getWriter().write("Un Permission");
```

```
    }
```

```
}
```

- Cấu hình SecurityFilterChain cho dự án “Authentication” và “Authorization” như sau:

```

import com.ra.constants.RoleName;
import com.ra.security.error.AccessDenied;
import com.ra.security.error.JwtEntryPoint;
import com.ra.security.jwt.JwtTokenFilter;

```



```

import com.ra.security.user_principal.UserDetailServiceCustom;
import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor // dùng để tiêm sự phụ thuộc dependencies injections
public class SecurityConfig {
    private final UserDetailServiceCustom userDetailServiceCustom;
    private final JwtTokenFilter jwtTokenFilter;
    private final JwtEntryPoint jwtEntryPoint;
    private final AccessDenied accessDenied;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(AbstractHttpConfigurer::disable)
            .authorizeHttpRequests(
                auth -> auth.requestMatchers("/api/v1/auth/**").permitAll()
                    .requestMatchers("/api/v1/admin/**").hasAuthority(String.valueOf(RoleName.ROLE_ADMIN))
                    .requestMatchers("/api/v1/user/**").hasAuthority(String.valueOf(RoleName.ROLE_USER))
                    .anyRequest().authenticated()
            )
            .authenticationProvider(authenticationProvider())
            .exceptionHandling(
                exception -> exception
                    .authenticationEntryPoint(jwtEntryPoint)
                    .accessDeniedHandler(accessDenied)
            )
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .addFilterBefore(jwtTokenFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setPasswordEncoder(passwordEncoder());
        provider.setUserDetailsService(userDetailServiceCustom);
        return provider;
    }
}

```

Bước 9: Tạo các đối tượng dto (data transfer object) như **UserLogin**, **UserRegister** là trong package **request**, **response** chúng ta sẽ có **JwtResponse** như sau:

- UserLogin:

```

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@NoArgsConstructor
@AllArgsConstructor
@Data
@Builder

```



```

public class UserLogin {
    private String email;
    private String password;
}

• UserRegister:
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.util.Set;

@NoArgsConstructor
@AllArgsConstructor
@Data
@Builder
public class UserRegister {
    private String fullName;
    private String email;
    private String password;
    private Set<String> roles;
}

• JwtResponse:
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.util.Set;

@NoArgsConstructor
@AllArgsConstructor
@Data
@Builder
public class JwtResponse {
    private String accessToken;
    private final String type = "Bearer";
    private String fullName;
    private String email;
    private String address;
    private String phone;
    private Set<String> roles;
}

```

Bước 10: Tạo các service của từng đối tượng như RoleService và UserService như sau:

```

• IRoleService:
import com.ra.constants.RoleName;
import com.ra.model.Roles;

public interface IRoleService {
    Roles findByRoleName(RoleName roleName);
}

• IUserService:
import com.ra.dto.request.UserLogin;
import com.ra.dto.request.UserRegister;
import com.ra.dto.response.JwtResponse;

public interface IUserService {
    void register(UserRegister userRegister);

    JwtResponse login(UserLogin userLogin);
}

• RoleServiceImpl:
import com.ra.constants.RoleName;
import com.ra.model.Roles;
import com.ra.repository.IRoleRepository;
import com.ra.service.IRoleService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class RoleServiceImpl implements IRoleService {
    private final IRoleRepository roleRepository;
}

```

```

@Override
public Roles findByRoleName(RoleName roleName) {
    return roleRepository.findByRoleName(roleName).orElseThrow(() -> new RuntimeException("role not found"));
}
}

• UserServiceImpl:
import com.ra.constants.RoleName;
import com.ra.dto.request.UserLogin;
import com.ra.dto.request.UserRegister;
import com.ra.dto.response.JwtResponse;
import com.ra.model.Roles;
import com.ra.model.Users;
import com.ra.repository.IUserRepository;
import com.ra.security.jwt.JwtProvider;
import com.ra.security.user_principal.UserPrincipal;
import com.ra.service.IRoleService;
import com.ra.service.IUserService;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.util.HashSet;
import java.util.Set;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class UserServiceImpl implements IUserService {
    private final IUserRepository userRepository;
    private final IRoleService roleService;
    private final PasswordEncoder passwordEncoder;
    private final AuthenticationProvider authenticationProvider;
    private final JwtProvider jwtProvider;

    @Override
    public void register(UserRegister userRegister) {
        Set<Roles> roles = new HashSet<>();
        if (userRegister.getRoles().isEmpty()) {
            roles.add(roleService.findByRoleName(RoleName.ROLE_USER));
        } else {
            userRegister.getRoles().forEach(
                role -> {
                    switch (role) {
                        case "admin":
                            roles.add(roleService.findByRoleName(RoleName.ROLE_ADMIN));
                        case "user":
                            roles.add(roleService.findByRoleName(RoleName.ROLE_USER));
                            break;
                        default:
                            throw new RuntimeException("role not found");
                    }
                }
            );
        }
        Users users = Users.builder()
            .fullName(userRegister.getFullName())
            .email(userRegister.getEmail())
            .password(passwordEncoder.encode(userRegister.getPassword()))
            .roles(roles)
            .build();
        userRepository.save(users);
    }

    @Override
    public JwtResponse login(UserLogin userLogin) {
        Authentication authentication;
        try {
            authentication
            = authenticationProvider.authenticate(new UsernamePasswordAuthenticationToken(userLogin.getEmail(), userLogin.ge

```

```

    } catch (AuthenticationException e) {
        throw new RuntimeException("Username or Password is incorrect");
    }
    UserPrincipal userPrincipal = (UserPrincipal) authentication.getPrincipal();
    return JwtResponse.builder()
        .accessToken(jwtProvider.generateToken(userPrincipal))
        .fullName(userPrincipal.getFullName())
        .email(userPrincipal.getEmail())
        .address(userPrincipal.getAddress())
        .phone(userPrincipal.getPhone())

        .roles(userPrincipal.getAuthorities().stream().map(GrantedAuthority::getAuthority).collect(Collectors.toSet()))
        .build();
    }
}

```

Bước 11: Tạo class AuthController dùng để định nghĩa các đường dẫn Đăng ký và Đăng nhập như sau:

```

import com.ra.dto.request.UserLogin;
import com.ra.dto.request.UserRegister;
import com.ra.dto.response.JwtResponse;
import com.ra.service.IUserService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

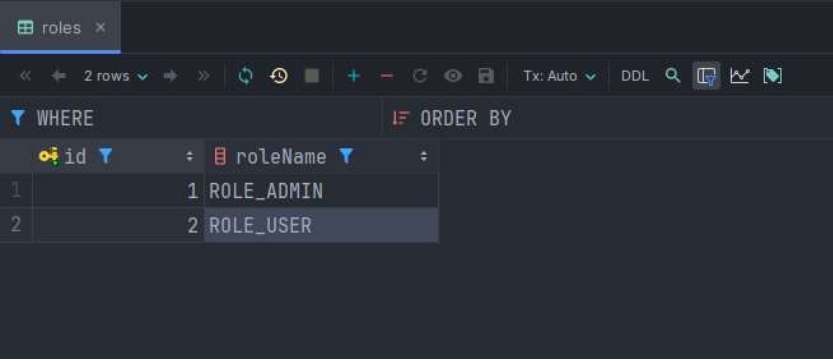
@RestController
@RequestMapping("/api/v1/auth")
@RequiredArgsConstructor
public class AuthController {
    private final IUserService userService;

    @PostMapping("/login")
    public ResponseEntity<JwtResponse> handleLogin(@RequestBody UserLogin userLogin) {
        return new ResponseEntity<>(userService.login(userLogin), HttpStatus.OK);
    }

    @PostMapping("/register")
    public ResponseEntity<String> handleRegister(@RequestBody UserRegister userRegister) {
        userService.register(userRegister);
        return new ResponseEntity<>("Register successfully", HttpStatus.CREATED);
    }
}

```

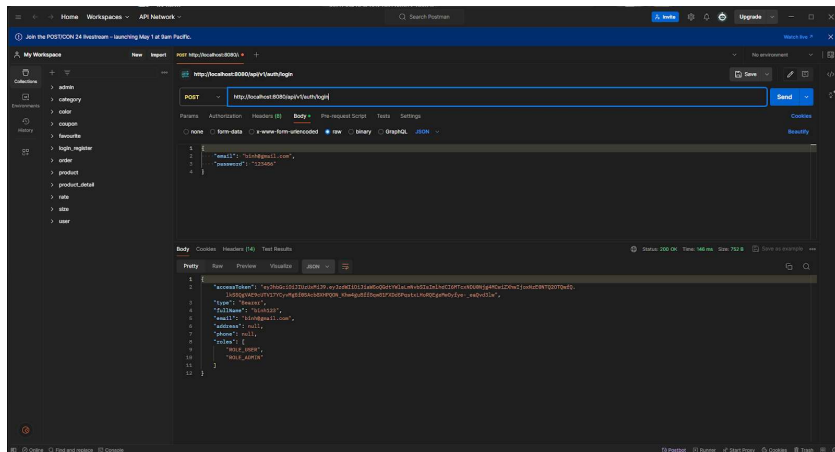
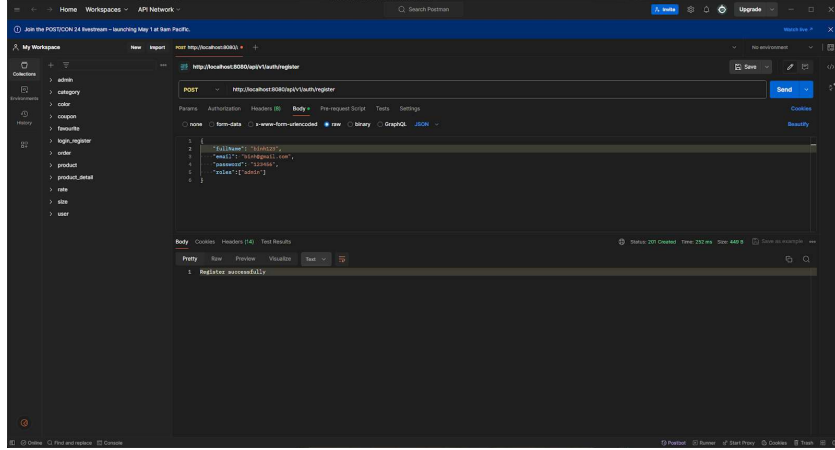
Bước 12: Thêm dữ liệu sẵn roles vào database như sau:



The screenshot shows a database management interface with a table named 'roles'. The table has two columns: 'id' and 'roleName'. There are two rows of data: one with id '1' and roleName 'ROLE_ADMIN', and another with id '2' and roleName 'ROLE_USER'.

	id	roleName
1	1	ROLE_ADMIN
2	2	ROLE_USER

Vì chúng ta sẽ ko có tính năng thêm role hay xóa role nên chúng ta sẽ thêm tay



Chạy chương trình và test chức năng đăng ký đăng nhập như yêu cầu của đề và kiểm nhiệm

Danh sách các bài học

